ESTRUCTURA DE DATOS · PLANTILLA DEL INFORME · C2

Tux Store v2 · Informe técnico

Evaluación por competencias C2 · NRC 42964 (lunes)

Asignatura	Estructura de Datos (24UC00428)
Docente	Carlos del Carpio
NRC · Grupo	42964 · Grupo B
Servidor	pinguino-emperador
Repositorio	https://github.com/72672623-cmyk/tuxstore-42964-grupoB
Versión del informe	v1.0 (E3)
Periodo	2026-20

Integrantes

Integrante	Usuario de GitHub	Rol / Requisitos a cargo
FERNÁNDEZ RAMÍREZ PATRICIO MICHAEL	72672623-cmyk	Líder (scrum) · Coordinación general
TICONA PACCO ALEX ROYER	Royer20061	Dev · catálogo · R1 y R5
MACHACA APAZA EDSON JAVIER	Edson-JMA	Dev · descargas · R2 y R3
FLORES CASTILLO BASILER EMANUEL	Basilercon	Dev · disco y archivo · R4 y R6
QUISPE MOLLENIDO JUAN DIEGO	61161913-jpg	QA y documentación · R7, pruebas

Capítulo 1 · Análisis del problema

1.1 Descripción del problema [E1]

Tux Store es un simulador de consola en C++ que imita a un gestor de paquetes de Linux (como apt). Lo usamos para comprender el funcionamiento interno de las estructuras de datos

dinámicas. El sistema resuelve la gestión de un catálogo de aplicaciones (mediante una lista enlazada), la atención ordenada de descargas respetando prioridades (usando una cola de prioridad), y el control del espacio en disco emulando un historial (usando una pila que registra cada bloque instalado para poder deshacer cambios).

1.2 Variables y condiciones [E1]

Variable	Tipo en C++	Unidad	Valor inicial o rango	Condición o regla que la afecta
VELOCIDAD	const int	MB/s	6	Regla 2 (Determina el tiempo de descarga)
DISCO	const int	MB	310	Regla 4 (Límite máximo de almacenamiento)
reloj	int	segundos	0	Regla 3 (Avanza al finalizar cada atención)
libre	int	MB	310 (DISCO)	Reglas 4 y 7 (Disminuye al instalar, aumenta al deshacer)
ultimaLlegada	int	segundos	0	Regla 5 (Valida que el tiempo no retroceda)
prioridad	int	entero	1 al 3	Regla 6 (1 sale primero, en empates aplica FIFO)

1.3 Requerimientos funcionales y no funcionales [E1]

ID	Requerimiento funcional	Orden(es)	Estructura
RF1	Gestionar catálogo de aplicaciones	catalogo, buscar, info, agregar	Lista enlazada
RF2	Añadir aplicaciones a la fila de descargas	instalar, fila	Cola de prioridad
RF3	Procesar descargas pendientes	procesar	Cola de prioridad y Pila

ID	Requerimiento funcional	Orden(es)	Estructura
RF4	Controlar el historial del disco y deshacer	historial, deshacer, espacio	Pila
RF5	Modificar prioridades y retirar apps	prioridad, retirar	Lista enlazada
RF6	Guardar y cargar estado (Persistencia)	guardar, cargar	Archivo txt, Lista y Pila
RF7	Mostrar reporte y salir liberando memoria	reporte, salir	Todas

ID	Requerimiento no funcional (cómo debe comportarse)
RNF1	Ninguna entrada inválida (errores de tipeo o parámetros incorrectos) cierra el programa.
RNF2	El código no usa contenedores de la biblioteca estándar (list, vector, queue, stack).
RNF3	Al salir (orden exit) se debe liberar toda la memoria dinámica con delete (imprimiendo "Nodos liberados").

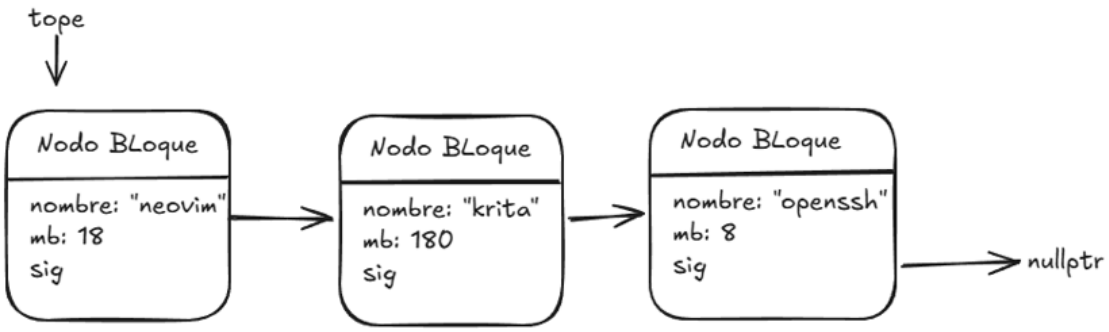
1.4 Estructuras propuestas y justificación [E1]

- **Catálogo (Lista Enlazada Simple):** Es la adecuada porque la cantidad de aplicaciones puede crecer dinámicamente al usar la orden agregar. Permite buscar recorriendo los nodos e insertar y retirar aplicaciones sin necesidad de reasignar bloques de memoria contigua.
- **Fila de Descargas (Cola de Prioridad):** Fundamental para cumplir la Regla 6. No sirve una pila porque las descargas no son LIFO, y no sirve una cola simple (FIFO puro) porque los procesos de prioridad 1 deben saltarse la fila y atenderse antes que los de prioridad 3.
- **Disco (Pila):** Es ideal porque emula un historial de acciones donde la función deshacer exige eliminar estrictamente el último bloque que se instaló (Last In, First Out). Una cola no serviría porque borraría el bloque más antiguo.

Capítulo 2 · Diseño de la solución

2.1 Estructuras y operaciones [E1]

Estructura	Operación	Orden	Qué punteros cambian
Lista	insertar al final	agregar	catalogo (si está vacía) o el puntero sig del último nodo
Lista	eliminar	retirar	catalogo (si es el primero) o el sig del nodo anterior
Cola	encolar con prioridad	instalar	frente, final, y el sig del nodo anterior a la posición de inserción
Cola	desencolar	procesar	frente (avanza al siguiente), final (se vuelve nullptr si queda vacía)
Pila	push	procesar	tope (pasa a apuntar al nuevo bloque)
Pila	pop	deshacer	tope (retrocede al siguiente nodo hacia el fondo)



2.2 Pseudocódigo [E1]

```
ALGORITMO instalar(app, seg)
  SI app existe Y app.estado == DISPONIBLE Y seg >= ultimaLlegada
ENTONCES
     CREAR nuevo NodoFila con app, seg y app.prioridad
     SI fila vacia O nuevo.prioridad mejor que frente.prioridad
ENTONCES
          insertar al inicio
     SINO
```

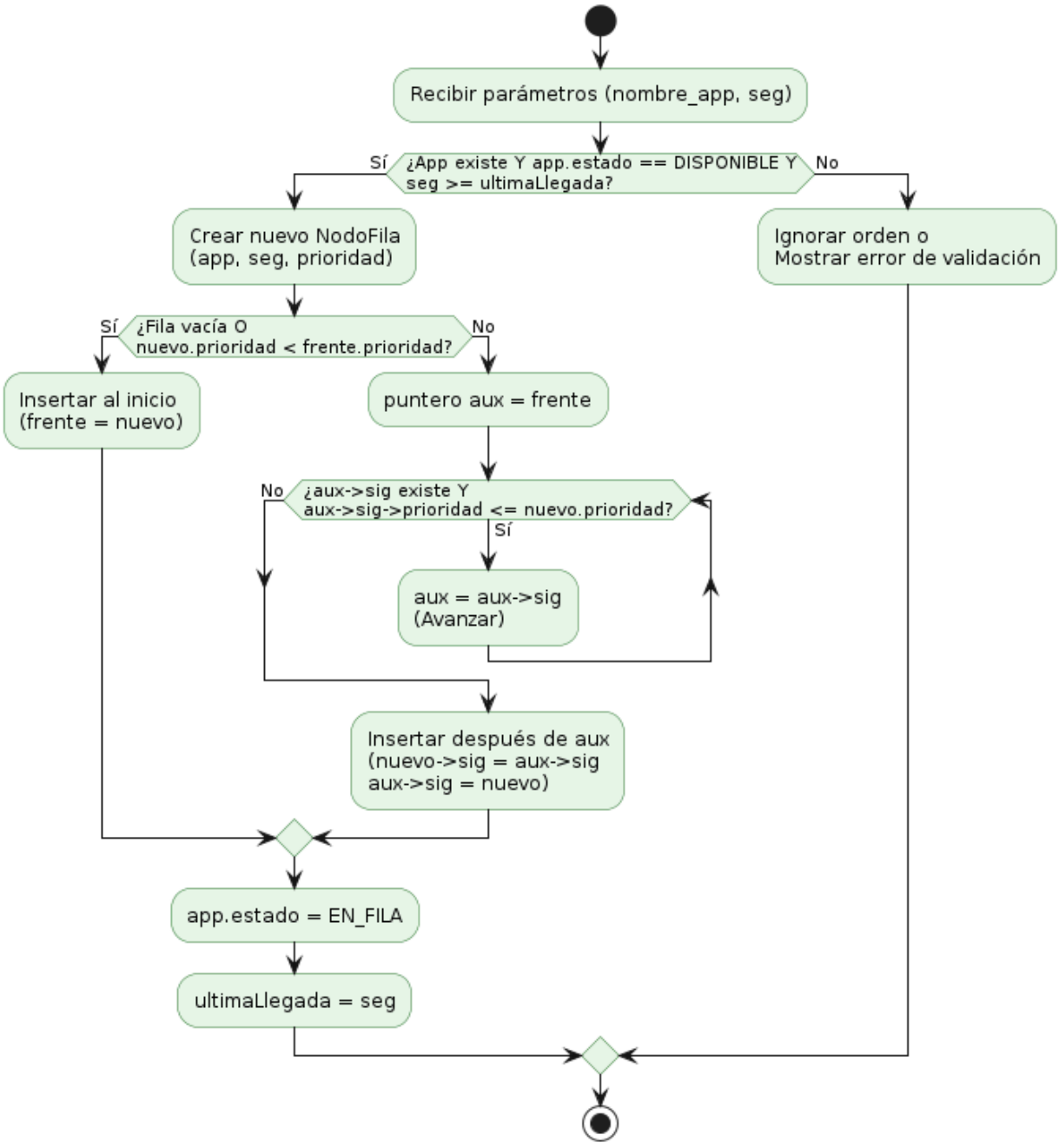
```
          avanzar mientras siguiente nodo tenga mejor o igual
prioridad
          insertar en esa posicion
      FIN SI
      app.estado = EN_FILA
      ultimaLlegada = seg
  FIN SI
FIN

ALGORITMO procesar()
  SI fila NO esta vacia ENTONCES
      nodo = frente de la cola
      app = buscarApp(nodo.nombre)
      SI app.mb > espacio_libre ENTONCES
          mostrar error
          app.estado = DISPONIBLE
      SINO
          calcular descarga, inicio, fin, espera
          reloj = fin
          espacio_libre = espacio_libre - app.mb
          app.estado = INSTALADA
          CREAR nuevo bloque de disco y hacer PUSH en tope
      FIN SI
      avanzar frente y hacer DELETE de nodo
  FIN SI
FIN
```

2.3 Diagramas de flujo [E2]

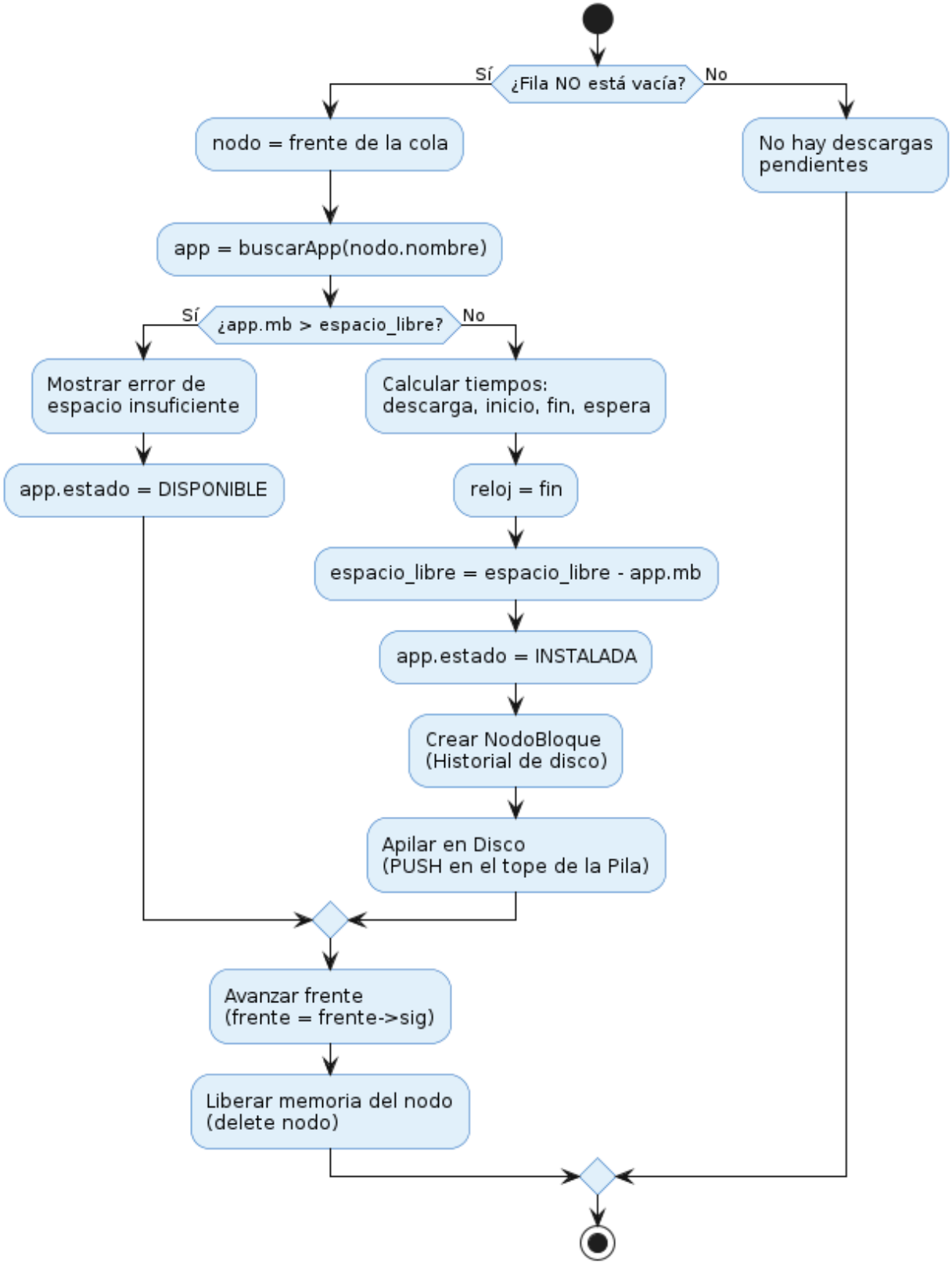
1. Diagrama de flujo para `instalar` (Encolar con prioridad)

Diagrama de Flujo: Función instalar(app, seg)



2. Diagrama de flujo para `procesar` (Desencolar y apilar)

Diagrama de Flujo: Función procesar()



2.4 Traza a mano de la secuencia oficial [E1]

Basado en el servidor pinguino-emperador (Velocidad: 6 MB/s, Disco: 310 MB)

#	App	Prior.	Pedida	MB	Descarga	Inicio	Fin	Espera	Libres
3	neofetch	3	0	-	-	-	-	-	-
4	neovim	2	1	-	-	-	-	-	-
5	openssh	1	3	-	-	-	-	-	-
7	openssh	1	3	8	2	3	5	0	302
8	vlc	3	4	-	-	-	-	-	-
10	krita	1	8	-	-	-	-	-	-
11	firefox	2	11	-	-	-	-	-	-
13	krita	1	8	180	30	8	38	0	122
14	neovim	2	1	18	3	38	41	37	104
15	firefox	2	11	210	-	-	-	-	104 (Error de espacio)
16	neofetch	3	0	1	1	41	42	41	103

Fila después de la orden 6: frente -> openssh (1) -> neovim (2) -> neofetch (3) -> final
Fila después de la orden 12: frente -> krita (1) -> neovim (2) -> firefox (2) -> neofetch (3) -> vlc (3) -> final

2.5 Justificación del diseño [E2]

- **Validación del tiempo:** Se usa la variable ultimaLlegada global para impedir que el usuario ingrese un segundo menor al del pedido anterior.
- **Cálculo de descarga:** Se usó la operación matemática con enteros (mb + VELOCIDAD - 1) / VELOCIDAD para asegurar el redondeo hacia arriba correcto según exige la regla.
- **Recuperación del disco al cargar:** Para que la pila conserve el mismo orden que tenía al guardarse (tope original), durante la orden cargar los bloques tipo "B" leídos del archivo no se insertan con PUSH normal al inicio, sino que se enlazan al FONDO de la pila.

Capítulo 3 · Desarrollo de la solución

3.1 Código fuente comentado [E3]

[NOTA PARA EL EQUIPO: Agreguen capturas breves del código de sus requerimientos o copien un par de funciones clave aquí indicando quién lo hizo]

3.2 Estructuras de control utilizadas [E3]

Fragmento (requisito)	Estructura de control	Para qué se usa	Condición de parada
Recorrido del catálogo (R1)	while	Mostrar todas las apps del catálogo	p == nullptr
Inserción con prioridad (R2)	while	Avanzar en la cola hasta encontrar una app con menor prioridad	p->sig == nullptr || p->sig->prioridad > prioridad
Lectura de tuxstore.txt (R6)	while	Leer el archivo línea por línea hasta el final	Fallo del flujo f >> tipo (Fin de archivo)
Validaciones de error	if / else if	Verificar que la entrada, app y estado sean correctos	—

3.3 Costo de las operaciones [E3]

Operación	Nodos que recorre (peor caso)	Explicación breve
agregar (al final)	O(n) (apps del catálogo)	Se debe recorrer toda la lista dos veces: una para evitar duplicados y otra para llegar al último nodo e insertar.
instalar (encolar)	O(k) (tamaño de la cola)	En el peor caso (app con prioridad 3) debe recorrer toda la cola actual para insertarse al final.
procesar	O(1)	Accede directamente al frente de la cola para desencolar y al tope de la pila para apilar el bloque.
deshacer	O(1)	Solo retira el nodo al que apunta el puntero tope de la pila.
retirar	O(n)	Requiere recorrer la lista buscando el nombre exacto de la aplicación usando

Operación	Nodos que recorre (peor caso)	Explicación breve
		punteros auxiliar ant y act.

3.4 Capturas de validación y pruebas [E3]

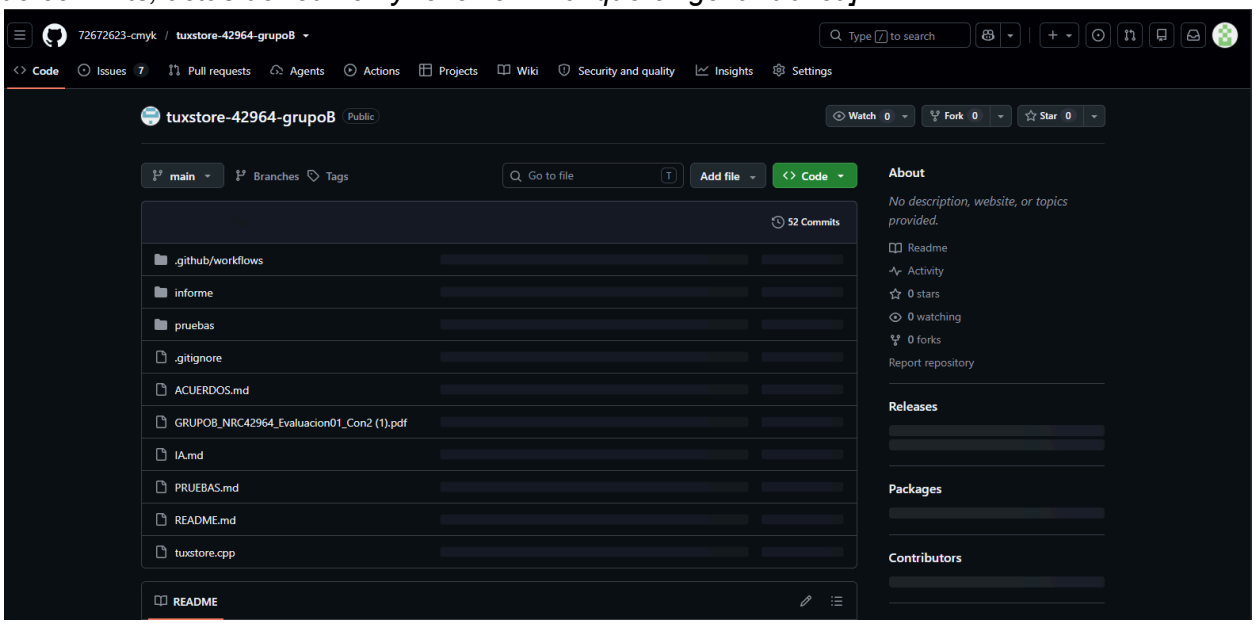
[NOTA PARA EL EQUIPO: Peguen aquí las capturas de la consola ejecutando su secuencia oficial (salida.txt) y la tabla de PRUEBAS.md]

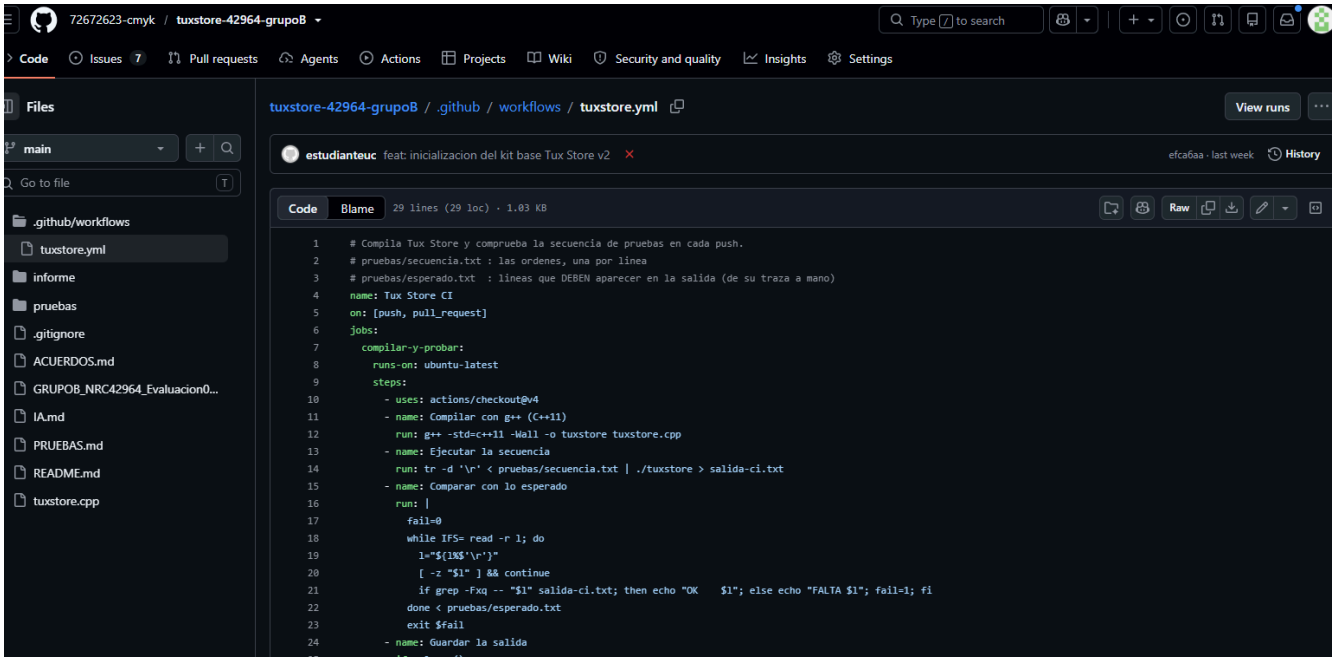
3.5 Manual de usuario [E3]

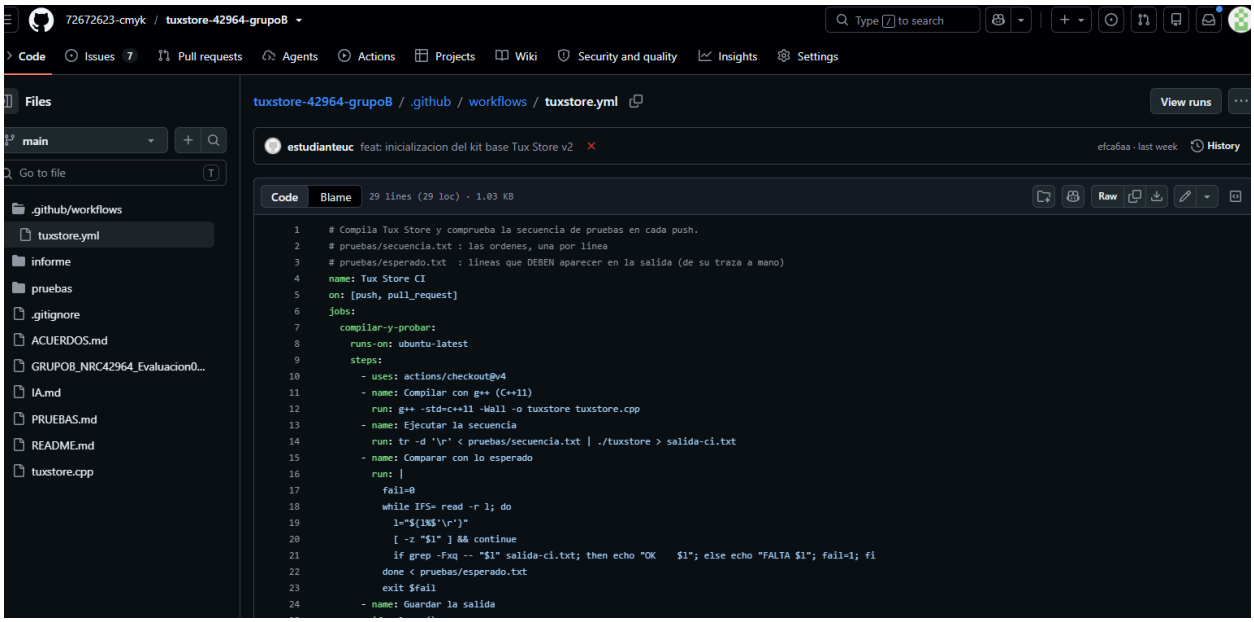
[NOTA PARA EL EQUIPO: Redactar brevemente cómo ejecutar el programa y 2 o 3 ejemplos de uso]

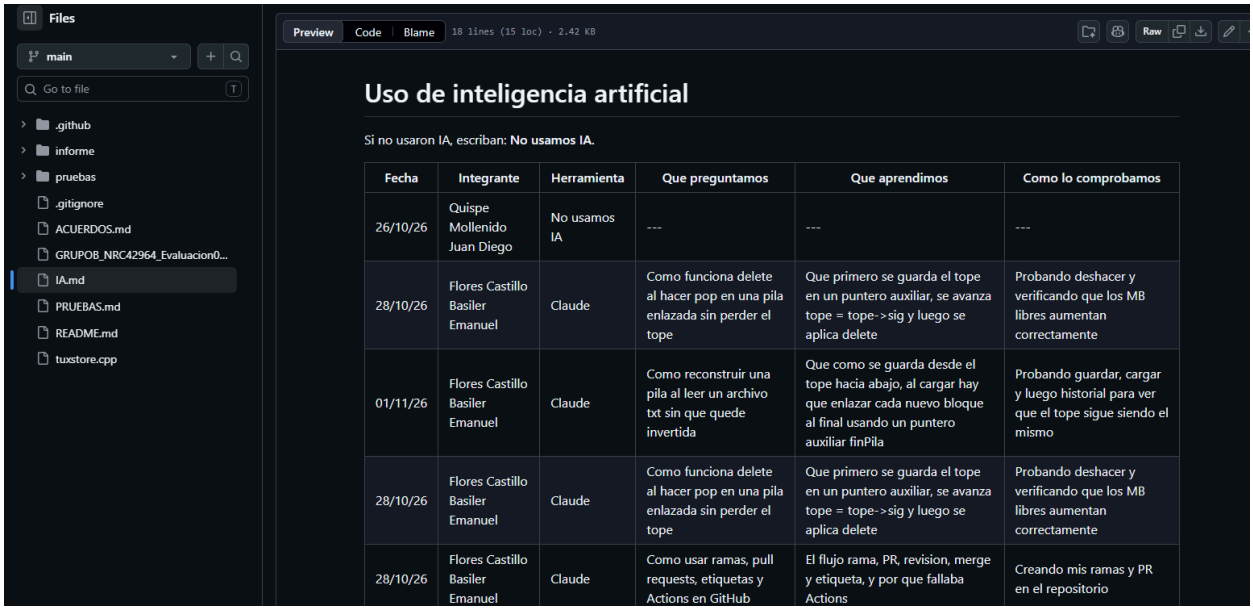
Capítulo 4 · Evidencias del trabajo en equipo

[NOTA PARA EL EQUIPO: Completen los apartados 4.1 a 4.5 con los enlaces a Github, la tabla de commits, actas de reunión y reflexión final que exige la rúbrica]




72672623-cmyk / tuxstore-42964-grupoB
Type / to search
Code
Issues 7
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
Files
main
Go to file
.github/workflows
tuxstore.yml
informe
pruebas
.gitignore
ACUERDOS.md
GRUPOB_NRC42964_Evaluacion0...
IA.md
PRUEBAS.md
README.md
tuxstore.cpp
tuxstore-42964-grupoB / .github / workflows / tuxstore.yml
View runs
estudianteuc feat: inicializacion del kit base Tux Store v2
efca6aa · last week
History
Code
Blame
29 lines (29 loc) · 1.03 KB
Raw
Compila Tux Store y comprueba la secuencia de pruebas en cada push.
pruebas/secuencia.txt : las ordenes, una por linea
pruebas/esperado.txt : lineas que DEBEN aparecer en la salida (de su traza a mano)
name: Tux Store CI
on: [push, pull_request]
jobs:
compilar-y-probar:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- name: Compilar con g++ (C++11)
run: g++ -std=c++11 -Wall -o tuxstore tuxstore.cpp
- name: Ejecutar la secuencia
run: tr -d '\r' < pruebas/secuencia.txt | ./tuxstore > salida-ci.txt
- name: Comparar con lo esperado
run: |
fail=0
while IFS= read -r l; do
l="${l%$'\r'}"
[-z "$l"] && continue
if grep -Fxq -- "$l" salida-ci.txt; then echo "OK $l"; else echo "FALTA $l"; fail=1; fi
done < pruebas/esperado.txt
exit $fail
- name: Guardar la salida


72672623-cmyk / tuxstore-42964-grupoB
Type / to search
Code
Issues 7
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
Files
main
Go to file
.github/workflows
tuxstore.yml
informe
pruebas
.gitignore
ACUERDOS.md
GRUPOB_NRC42964_Evaluacion0...
IA.md
PRUEBAS.md
README.md
tuxstore.cpp
tuxstore-42964-grupoB / .github / workflows / tuxstore.yml
View runs
estudianteuc feat: inicializacion del kit base Tux Store v2
efca6aa · last week
History
Code
Blame
29 lines (29 loc) · 1.03 KB
Raw
Compila Tux Store y comprueba la secuencia de pruebas en cada push.
pruebas/secuencia.txt : las ordenes, una por linea
pruebas/esperado.txt : lineas que DEBEN aparecer en la salida (de su traza a mano)
name: Tux Store CI
on: [push, pull_request]
jobs:
compilar-y-probar:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- name: Compilar con g++ (C++11)
run: g++ -std=c++11 -Wall -o tuxstore tuxstore.cpp
- name: Ejecutar la secuencia
run: tr -d '\r' < pruebas/secuencia.txt | ./tuxstore > salida-ci.txt
- name: Comparar con lo esperado
run: |
fail=0
while IFS= read -r l; do
l="${l%$'\r'}"
[-z "$l"] && continue
if grep -Fxq -- "$l" salida-ci.txt; then echo "OK $l"; else echo "FALTA $l"; fail=1; fi
done < pruebas/esperado.txt
exit $fail
- name: Guardar la salida



Uso de inteligencia artificial

Si no usaron IA, escriban: **No usamos IA.**

Fecha	Integrante	Herramienta	Que preguntamos	Que aprendimos	Como lo comprobamos
26/10/26	Quispe Mollenido Juan Diego	No usamos IA	---	---	---
28/10/26	Flores Castillo Basiler Emanuel	Claude	Como funciona delete al hacer pop en una pila enlazada sin perder el tope	Que primero se guarda el tope en un puntero auxiliar, se avanza tope = tope->sig y luego se aplica delete	Probando deshacer y verificando que los MB libres aumentan correctamente
01/11/26	Flores Castillo Basiler Emanuel	Claude	Como reconstruir una pila al leer un archivo txt sin que quede invertida	Que como se guarda desde el tope hacia abajo, al cargar hay que enlazar cada nuevo bloque al final usando un puntero auxiliar finPila	Probando guardar, cargar y luego historial para ver que el tope sigue siendo el mismo
28/10/26	Flores Castillo Basiler Emanuel	Claude	Como funciona delete al hacer pop en una pila enlazada sin perder el tope	Que primero se guarda el tope en un puntero auxiliar, se avanza tope = tope->sig y luego se aplica delete	Probando deshacer y verificando que los MB libres aumentan correctamente
28/10/26	Flores Castillo Basiler Emanuel	Claude	Como usar ramas, pull requests, etiquetas y Actions en GitHub	El flujo rama, PR, revision, merge y etiqueta, y por que fallaba Actions	Creando mis ramas y PR en el repositorio



ESTRUCTURA DE DATOS · PLANTILLA DEL INFORME · C2

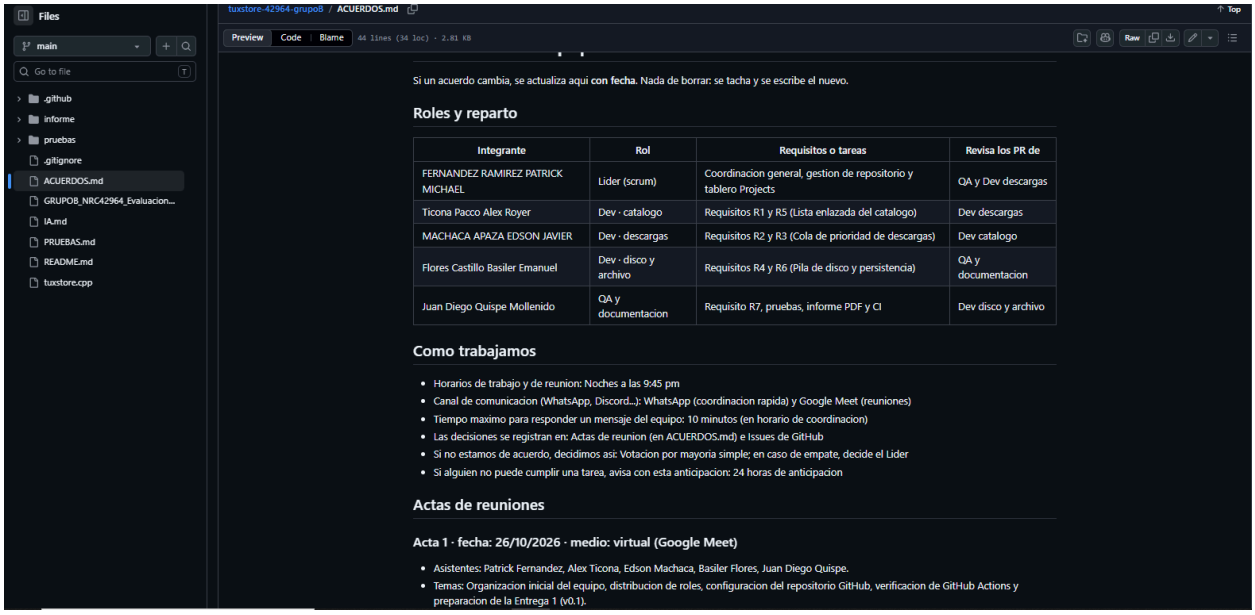
Tux Store v2 · Informe técnico

Evaluación por competencias C2 · NRC 42964 (lunes)

Asignatura	Estructura de Datos (24UC00428)
Docente	Carlos del Carpio
NRC · Grupo	42964 · Grupo B
Servidor	pinguino-emperador
Periodo	2026-20

Integrantes

Nombre	Usuario GitHub	Rol
FERNANDEZ RAMIREZ PATRICK MICHAEL	72672623-cmyk	Líder (scrum)
Ticona Pacco Alex Royer	Royer20061	Dev catálogo
MACHACA APAZA EDSON JAVIER	Edson-Machaca	Dev descargas
Flores Castillo Basiler Emanuel	Basilercon	Dev disco y archivo
QUISPE MOLLENIDO JUAN DIEGO	61161913-jpg	QA y documentación



Si un acuerdo cambia, se actualiza aqui **con fecha**. Nada de borrar: se tacha y se escribe el nuevo.

Roles y reparto

Integrante	Rol	Requisitos o tareas	Revisa los PR de
FERNANDEZ RAMIREZ PATRICK MICHAEL	Lider (scrum)	Coordinacion general, gestion de repositorio y tablero Projects	QA y Dev descargas
Ticona Pacco Alex Royer	Dev - catalogo	Requisitos R1 y R5 (Lista enlazada del catalogo)	Dev descargas
MACHACA APAZA EDSON JAVIER	Dev - descargas	Requisitos R2 y R3 (Cola de prioridad de descargas)	Dev catalogo
Flores Castillo Basiler Emanuel	Dev - disco y archivo	Requisitos R4 y R6 (Pila de disco y persistencia)	QA y documentacion
Juan Diego Quispe Mollenido	QA y documentacion	Requisito R7, pruebas, informe PDF y CI	Dev disco y archivo

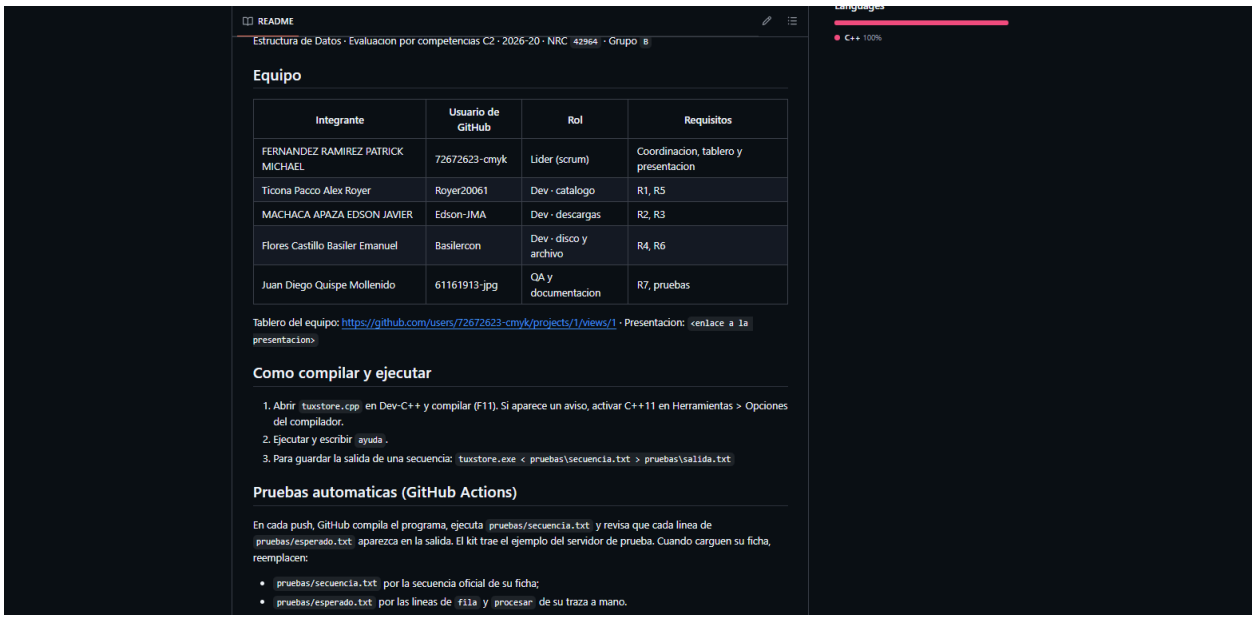
Como trabajamos

- Horarios de trabajo y de reunion: Noches a las 9:45 pm
- Canal de comunicacion (WhatsApp, Discord...): WhatsApp (coordinacion rapida) y Google Meet (reuniones)
- Tiempo maximo para responder un mensaje del equipo: 10 minutos (en horario de coordinacion)
- Las decisiones se registran en: Actas de reunion (en ACUERDOS.md) e Issues de GitHub
- Si no estamos de acuerdo, decidimos asi: Votacion por mayoria simple; en caso de empate, decide el Lider
- Si alguien no puede cumplir una tarea, avisa con esta anticipacion: 24 horas de anticipacion

Actas de reuniones

Acta 1 - fecha: 26/10/2026 - medio: virtual (Google Meet)

- Asistentes: Patrick Fernandez, Alex Ticona, Edson Machaca, Basiler Flores, Juan Diego Quispe.
- Temas: Organizacion inicial del equipo, distribucion de roles, configuracion del repositorio GitHub, verificacion de GitHub Actions y preparacion de la Entrega 1 (v0.1).



Estructura de Datos · Evaluacion por competencias C2 · 2026-20 · NRC `42964` · Grupo `8`

Equipo

Integrante	Usuario de GitHub	Rol	Requisitos
FERNANDEZ RAMIREZ PATRICK MICHAEL	72672623-cmyk	Lider (scrum)	Coordinacion, tablero y presentacion
Ticona Pacco Alex Royer	Royer20061	Dev - catalogo	R1, R5
MACHACA APAZA EDSON JAVIER	Edson-JMA	Dev - descargas	R2, R3
Flores Castillo Basiler Emanuel	Basilercon	Dev - disco y archivo	R4, R6
Juan Diego Quispe Mollenido	61161913-jpg	QA y documentacion	R7, pruebas

Tablero del equipo: https://github.com/users/72672623-cmyk/projects/1/views/1 · Presentacion: `<enlace a la presentacion>`

Como compilar y ejecutar

1. Abrir `tuxstore.cpp` en Dev-C++ y compilar (F11). Si aparece un aviso, activar C++11 en Herramientas > Opciones del compilador.
2. Ejecutar y escribir `ayuda`.
3. Para guardar la salida de una secuencia: `tuxstore.exe < pruebas\secuencia.txt > pruebas\salida.txt`

Pruebas automaticas (GitHub Actions)

En cada push, GitHub compila el programa, ejecuta `pruebas/secuencia.txt` y revisa que cada linea de `pruebas/esperado.txt` aparezca en la salida. El kit trae el ejemplo del servidor de prueba. Cuando carguen su ficha, reemplacen:

- `pruebas/secuencia.txt` por la secuencia oficial de su ficha;
- `pruebas/esperado.txt` por las lineas de `fila` y `procesar` de su traza a mano.